

FedMPS: A Robust Differential Privacy Federated Learning Based on Local Model Partition and Sparsification for Heterogeneous IIoT Data

Danxin Wang[✉], Yuyang Gao, Shanchen Pang[✉], Chen Zhang, Xiaoman Zhang, and Ming Li[✉]

Abstract—In the emerging Industrial Internet of Things (IIoT) applications, federated learning (FL) enables model training without the need to transmit raw data directly. Nevertheless, transmitting model parameters could still reveal private information. To further protect local model parameters, differential privacy combined with FL (DPFL) has been introduced. Nonetheless, adding noise in DPFL can severely impact model performance, especially in non-independent and identically distributed (non-iid) data scenarios typical of IIoT environments. It is necessary to carefully balance privacy preservation and utility. In this article, we propose a robust DPFL scheme leveraging local model partition and sparsification (namely, FedMPS) for heterogeneous IIoT scenarios. The local model is divided into a shared part, which is sparsified before adding noise to mitigate its impact, and a private part that remains on the client. We provide a theoretical analysis of the privacy guarantees. Extensive experiments on common datasets, including Fashion-MNIST, CIFAR-10, and CIFAR-100, demonstrate that the proposed approach achieves a better privacy–utility tradeoff, with a 10%–20% improvement compared to baseline methods, and performs well especially in non-iid scenarios.

Index Terms—Data heterogeneity, differential privacy federated learning (DPFL), Industrial Internet of Things (IIoT), privacy–utility tradeoff.

I. INTRODUCTION

THE RAPID development of the Industrial Internet of Things (IIoT) has introduced a large number of intelligent end devices and edge nodes that generate substantial amounts of real-time data daily, such as sensor readings, video streams, and operation logs [1], [2]. The analysis of these data is essential, as it provides key insights that support data-driven decision-making. This not only enhances production efficiency but also enables predictive maintenance to reduce downtime, improves product quality, and increases the overall reliability

of factories and systems. Artificial intelligence (AI) techniques are widely applied in IIoT for data-driven analysis, playing an essential role in areas, such as smart manufacturing, fault detection, and equipment monitoring. However, traditional centralized machine learning methods typically rely on aggregating data at a central server for unified training, which faces significant limitations in the IIoT. First, it is involved heavy communication and storage overhead that centralizing the huge volumes of IIoT data generated by widely dispersed devices. Furthermore, given that the IIoT often contains commercially valuable and potentially sensitive information, direct transmission to a central server poses substantial privacy leakage, potentially even violating data protection regulations.

Federated learning (FL) [3], [4], [5] has been proposed as a solution to these challenges, requiring only parameters of the local updated model to be uploaded to the central server, where these parameters are aggregated, thus facilitating distributed model training without exposing raw data. It is well suited for distributed IIoT scenarios which widely distributed edge nodes train locally with the uploaded data from resource constrained collection devices, then local model updates transfer to central server for aggregation.

While integrating FL with IIoT provides significant benefits for privacy protection and communication efficiency, it still faces the risk of privacy leakage. The transmission of local model parameters, which contain information derived from the local data generated by intelligent end devices, can still lead to unintended privacy leakage through advanced inference attacks [6]. To further enhance privacy protection in FL, differential privacy (DP) [7] is commonly applied. By introducing noise into the model updates, DP offers a mathematically rigorous framework that minimizes the risk of revealing details about any individual data point from the shared parameters. Previous research has shown that directly integrating DP into FL, such as using Gaussian mechanisms, ensures strong privacy [8], [9]. However, it is unavoidably causing performance degradation directly to add noise to the model parameters, particularly in deep learning models with complex architectures and large parameter sets. Therefore, several attempts [10], [11], [12], [13] have been made to reduce the perturbations implied by the noise, such as introducing regularization, improved optimizers, and sparsification. However, many of these approaches still apply noise to the entire model, and have yet to achieve satisfactory results in maintaining model accuracy while preserving privacy.

Received 14 November 2024; revised 10 January 2025; accepted 25 January 2025. Date of publication 29 January 2025; date of current version 9 May 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62402527, and in part by the Shandong Provincial Natural Science Foundation under Grant ZR2024MF129, Grant ZR2023QF101, and Grant ZR2023QF153. (Corresponding author: Shanchen Pang.)

Danxin Wang, Yuyang Gao, Shanchen Pang, Chen Zhang, and Xiaoman Zhang are with the Qingdao Institute of Software, College of Computer Science and Technology, Shandong Key Laboratory of Intelligent Oil and Gas Industrial Software, China University of Petroleum (East China), Qingdao 266580, China (e-mail: wangdx@upc.edu.cn; Z23070172@s.upc.edu.cn; pangsc@upc.edu.cn; zhangchen@upc.edu.cn; Z23070171@s.upc.edu.cn).

Ming Li is with Shandong Inspur Database Technology Company Ltd., Jinan 250000, China (e-mail: liming2017@inspur.com).

Digital Object Identifier 10.1109/IIOT.2025.3536035

Additionally, the IIoT environment presents a range of new challenges, especially the non-independent and identically distributed (non-iid) characteristics of data. Data from different end devices across various factories or production lines may vary substantially in quantity and distribution. For instance, in an IIoT quality inspection system, different factories have cameras capturing images with varying defect types, leading to non-iid data distributions across locations. The non-iid data leads to considerable discrepancies in model parameters during FL training. It negatively affects both the overall accuracy and the convergence speed of the global model, both of which are crucial in industrial production. Moreover, the impact of DP noise is amplified in such non-iid settings in IIoT, where the variability in client data exacerbates the effects of DP noise, leading to a more significant impact on model utility [14]. It is with regret that existing approaches often fail to balance privacy protection and model utility in heterogeneous data scenarios.

In response to these challenges, we proposed a robust differentially private FL method FedMPS for IIoT, to improve the privacy–utility tradeoff and the model’s personalized performance by introducing local model partitioning and sparsification techniques. We divide the local model into two parts: 1) a shared partition which is aggregated with noise to protect privacy and 2) a private partition which remains local to preserve personalization. Specifically, the shared partition is primarily composed of convolutional layers designed to extract input data features. Sharing this part enables collaboration with other clients to train a more robust feature extractor. On the other hand, the private partition consists of linear layers responsible for classifying features, and applying noise to this portion would significantly degrade model performance. Privatizing this part effectively reduces the impact of noise on the model, while improving the model’s personalized performance. Further, we apply a layer-wise sparsification to the shared partition of the updated model. By focusing on the most significant updates in each layer, sparsification further mitigates the interference caused by the noise, improving both the efficiency and performance of the DP federated learning (DPFL) process. Through rigorous analysis and extensive experimentation, we show that our method effectively improves the balance between protecting privacy and maintaining utility, outperforming existing approaches in DPFL, especially for non-iid scenarios.

This article presents the following key contributions.

- 1) We propose a novel DPFL architecture FedMPS, which achieves an effective balance between protecting privacy and maintaining utility by reducing the impact of noise perturbations through model partitioning and sparsification.
- 2) Our method improves model performance under non-iid data through local personalized training, making it highly appropriate for heterogeneous IIoT scenarios.
- 3) We present rigorous analysis and extensive experimentation on real-world datasets to show the effectiveness of our algorithm, which achieves a 10%–20% increase in accuracy compared to other methods without

compromising privacy guarantees, and performs well especially in non-iid scenarios.

The remaining part of this article proceeds as follows. In Section II, we present related works on FL in the IIoT, as well as DPFL and personalization FL. In Section III, we show the detail of our method. In Section IV, we introduce the system model. In Section V, we conduct an analysis of the privacy guarantees based on Rényi DP (RDP). The experimental results on real-world datasets are shown in Section VI. Finally, Section VII presents the conclusions.

II. RELATED WORKS

In this section, we present a comprehensive review of the existing researches on the application of FL in IIoT environments, as well as studies on DP mechanisms within FL and personalized FL frameworks.

A. Federated Learning in IIoT

FL has become popular in IIoT due to its distributed nature, which reduces communication overhead and enables local processing on edge devices. While this is advantageous for the vast and diverse data sources in IIoT, such as sensors, cameras, and drones, FL in IIoT still faces major challenges, particularly around privacy protection and non-iid data handling.

Several studies have proposed privacy-preserving mechanisms. For instance, Xu et al. [15] introduced a blockchain-based approach which improves both tamper-resistance and decentralization capabilities of IIoT devices. This method incorporates an adaptive DP mechanism alongside a node contribution consensus protocol to ensure data-sharing security in IIoT networks. Similarly, Putra et al. [16] proposed a secure parameter aggregation technique based on Proof-of-Authority (PoA) blockchain and Gaussian DP, aimed at bolstering trust, security, and privacy in FL for IIoT. In another approach, HFDPFL [17] presented a hierarchical fair FL framework with DP, integrating a reputation mechanism to promote fairness across devices. Meanwhile, for handling non-iid data in IIoT, FEDGS [18] proposed a hierarchical FL architecture that leverages the natural clustering of IIoT factory devices. However, few studies have simultaneously considered both privacy preservation and non-iid data issues in FL for IIoT.

B. Differential Privacy in Federated Learning

Alternative privacy-preserving methods, such as homomorphic encryption, secure multiparty computation, and blockchain, effectively protect privacy but typically come with substantial computational or communication costs. In contrast, DP, due to its low overhead, has become widely applied in FL. DP minimizes the impact of individual data changes on output, providing strong privacy protection in FL systems. This is particularly valuable in IIoT environments, where safeguarding sensitive data is essential. The privacy is quantified using (ϵ, δ) , where smaller ϵ enhance privacy but introduce more noise. In machine learning, Abadi et al. [19] first introduced the concept of DP to protect sensitive data. Later, it was

extended to FL by [9], where DP is applied to safeguard clients' private information throughout the training procedure.

Depending on how neighboring dataset is defined, the DP in FL is categorized into record-level DP and client-level DP. For record-level DP, the primary concern is to protect the privacy of individual data points in the dataset of each client. In this context, neighboring dataset refers to those that differ by the inclusion or exclusion of just one data point from the dataset. Alternatively, client-level DP aims to safeguard the privacy of each individual client, meaning that the neighboring dataset differs by the inclusion or exclusion of the entire local dataset of a client. In contrast, client-level DP provides more sufficient privacy protection, as it shields the identity of the client itself along with all their data [9], which is particularly practical in FL. Therefore, we adopt the client-level neighboring dataset definition in this work. To implement client-level DP effectively in FL, clipping local updates and adding noise before transmission to the server are required. While this approach enhances client privacy, it also introduces challenges, such as potential performance degradation and slower convergence.

To address this issue, Andrew et al. [20] introduced dynamic clipping thresholds based on quantiles, offering better adaptability, and Zhang et al. [21] leveraged entropy theory to present an adaptive DP approach. CAFL [22] fine-tunes noise application at a more fine-grained parameter level, offering novel perturbation techniques, and ACS-FL [23] refines this by randomizing parameter values within specific ranges, balancing noise application and accuracy. Some approaches focus on the improvement of the model, such as PPSGD [24], to improve performance without compromising privacy. FedDPA [25] adds an adaptive regularization strategy to enhance learning capacity and robustness against clipping. However, it still transmits and add noise on the whole model, which makes it more susceptible to the clipping operation and increase computational costs. PrivateFL [14] views noise as a form of heterogeneity, introducing a transformation layer to mitigate distribution shifts caused by noise. Other studies like BLUR [10], which employs local update regularization and sparsification, and FedSMP [13], which applies global sparsification, also enhance robustness to noise. Additionally, DP-MEMSGD [26] maintains the accuracy by remembering the unused portions of gradients. FL-CS-DP [27] utilizes discrete cosine transform (DCT) compression, though it faces limitations under certain conditions.

Despite various advancements in DPFL, existing methods still face significant performance degradation caused by the added DP noise. Furthermore, most current approaches fail to address the common non-iid problem, which leads to model divergence and amplifies the effects of DP noise, further degrading performance.

C. Personalized Federated Learning

The FedAvg algorithm [3] is a pioneering method in FL that aggregates locally trained models to generate a global model. Nevertheless, it has been reported that the non-iid characteristic of local data presents significant challenges,

often resulting in poor performance and inefficiencies in the global model when applied to diverse data distributions.

Various approaches have been suggested to address the challenges posed by non-iid data in federated settings. For instance, researches like Ditto [28] and MOON [29] have been conducted on using regularization techniques to enhance the stability of local updates in non-iid settings, but all clients still share the same model. Some methods, such as FedAMP [30] and FedALA [31], have focused on refining the aggregation process to better extract useful information from the global model in a personalized manner. While these approaches improve the balance between personalization and generalization, they often incur higher computational and communication overhead. Another line of research has focused on model partitioning strategies, as seen in [32], [33], and [34]. These methods divide the model into shared and personalized components, with the shared component aggregated across clients and the personalized component maintained independently by each client. For example, FedRep [32] partitions the model into a linear classifier and a feature extractor, where each client manages their classifier independently. Similarly, FedBABU [33] employs the same partitioning but refrains from training the classifier further post-initialization, opting instead for fine-tuning after aggregation. Similar approaches, such as FedProto [35] and FedGH [36], also explore model partitioning but differ in how they handle aggregation. FedProto [35] aggregates features from client data to guide local training, while FedGH [36] uploads both client labels and feature extractor outputs to train a global classifier. However, the direct upload of features may compromise client privacy.

In existing approaches, most efforts focus on addressing the non-iid characteristic of local data without considering the effect of privacy protection mechanisms. While these methods improve model performance under data heterogeneity, they often overlook the critical need for safeguarding sensitive data, especially in IIoT scenarios where privacy protection is essential, because devices handle highly sensitive operational data.

III. PRELIMINARY

A. Federated Learning

FL enables multiple clients to jointly train a global model in a distributed manner, without the need to share their private data. Given N clients, each client $i \in \{1, 2, \dots, N\}$ holds a local dataset D_i , and the union of all clients' datasets forms the total dataset $D = \bigcup_{i=1}^N D_i$. In each communication round t , a random subset of K clients is chosen to take part in the training process. Formally, the general optimization problem of FL can be described as follows:

$$\min_{\omega} \left\{ f(\omega, D) = \sum_{i \in N} p_i f_i(\omega, D_i) \right\} \quad (1)$$

$$f_i(\omega, D_i) = \mathbb{E}_{x \sim \mathcal{P}_i} [f_i(\omega, x)] \quad (2)$$

where w represents the model parameters and D_i denotes the local dataset of client i . Then, $f_i(w, D_i)$ refers to the local objective function. x is a sample from D_i , and $\mathcal{P}_i$ denotes its

data distribution. The weight assigned to client i is represented as p_i , which often proportional to the size of corresponding local dataset D_i .

To align with IIoT scenarios, this article examines a more practical FL setup, where the end devices data is non-iid, i.e., the $\mathcal{P}_i$ varies across the edge nodes.

Client-Side Training: During each communication round t , the selected client $i \in K$ using its dataset D_i to train the local model w_i^t . The local model update is performed via stochastic gradient descent (SGD) as follows:

$$w_i^{t+1} = w_i^t - \eta_t \nabla f_i(w_i^t, D_i) \quad (3)$$

where $\nabla f_i(w_i^t, D_i)$ is the gradient, and η_t is the learning rate at round t .

Server-Side Aggregation: After each participating client completes the local training process in round t , the trained local models $\{w_1^t, w_2^t, \dots, w_K^t\}$ are uploaded to a central server, where the local models are aggregated to form a new global model w^t as follows:

$$w^t = \sum_{i \in K} \frac{n_i}{n} w_i^t \quad (4)$$

where n_i denotes the size of the local dataset D_i for client i , and $n = \sum_{i \in K} n_i$ represents the size of all participating clients' datasets during communication round t .

B. Differential Privacy

DP is a commonly used approach aimed at safeguarding data privacy by perturbing the original data through adding noise. This process prevents attackers from extracting private individual-specific information from the released data. Formally, DP is defined as follows.

Definition 1 (DP [7]): A randomized algorithm $\mathcal{M}$ is said to satisfy (ϵ, δ) -DP if for any pair of neighboring datasets D and D' for any subset $S \subseteq \text{Range}(\mathcal{M})$, the following condition holds:

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S] + \delta \quad (5)$$

where ϵ is the privacy budget that controls the level of privacy guarantee, and δ is a small probability representing the allowed relaxation in privacy. The smaller the ϵ , the stricter the privacy protection.

This definition guarantees that the output of the randomized algorithm $\mathcal{M}$ remains nearly unchanged between neighboring datasets D and D' , which differ by only a single individual. Consequently, an adversary is hard to determine whether a specific individual's data is part of the dataset, thereby safeguarding the privacy of individuals within dataset D . In our work, we adopt client-level DP, meaning that neighboring datasets are defined by adding or removing the entire dataset of a client.

To quantify privacy loss over multiple iterations in FL more precisely, we leverage RDP [37] for our analysis, which is given by the following definition.

Definition 2 ((α, ρ)-RDP [37]): A randomized mechanism $\mathcal{M}$ is said to satisfy (α, ρ) -RDP for a given real number $\alpha > 1$

and privacy parameter $\rho \geq 0$, if for all neighboring datasets D and D' , the following inequality holds:

$$D_\alpha[\mathcal{M}(D) \parallel \mathcal{M}(D')] := \frac{1}{\alpha - 1} \log \mathbb{E} \left[\left(\frac{\mathcal{M}(D)}{\mathcal{M}(D')} \right)^\alpha \right] \leq \rho(\alpha). \quad (6)$$

RDP is an extension of classical DP that provides a more flexible privacy framework by leveraging the Rényi divergence, which generalizes the concept of divergence between probability distributions. It allows for more precise tracking of privacy loss, especially in iterative algorithms like FL. To translate RDP back into the classical DP framework, we can use the following lemma [38] to prove a randomized mechanism $\mathcal{M}$ satisfies $(\rho(\alpha) + (\log(1/\delta)/\alpha - 1), \delta)$ -DP, if it satisfies $(\alpha, \rho(\alpha))$ -RDP.

Additionally, several useful definitions and lemmas related to RDP and DP will be used to prove that our method satisfies DP throughout the following sections of this article. First, l_2 -sensitivity [7] is defined as $\psi(h) = \max \|h(D) - h(D')\|_2$, which represents the maximum change in the output of a query function h when applied to two neighboring datasets D, D' . Then, the Gaussian mechanism [37] is also introduced, a well-known method for achieving DP. When a query function h has an l_2 -sensitivity $\psi(h)$, the Gaussian mechanism $\mathcal{M} = h(D) + \mathcal{N}(0, \sigma^2 \psi(h)^2 \mathbf{I}_d)$ satisfies $(\alpha, \alpha/2\sigma^2)$ -RDP. Finally, the RDP subsampling lemma [38] demonstrates the amplification properties of RDP under subsampling and the RDP composition lemma [37] explains the compositional properties of RDP, showing that if $\mathcal{M}_1$ satisfies (α, ρ_1) -RDP and $\mathcal{M}_2$ satisfies (α, ρ_2) -RDP, then their combined mechanism, denoted by $\mathcal{M}_1 \circ \mathcal{M}_2$, satisfies $(\alpha, \rho_1 + \rho_2)$ -RDP.

IV. SYSTEM MODEL

Our system model adopts a standard FL architecture for IIoT scenarios comprising a central server, collaborative edge nodes, and resource-constrained collection devices, as shown in Fig. 1. Numerous data collection devices, such as temperature sensors, surveillance cameras, patrol drones, and assembly line robots, due to resource constrained, usually upload the collected data directly to trusted edge nodes with sufficient computation and communication resources to process. These edge nodes are capable of deploying models like deep neural networks, to perform model training on collected local data from end devices and upload the updates to the central server, which then aggregates them to form a new global model, later distributing it back to the edge nodes for further improvement. The details of primary components are described as follows.

Data Collection Devices: In IIoT scenarios, these devices usually are resource constrained and clustered in different trusted edge nodes. The connection between these devices and edge nodes is mostly through reliable wired or internal networks. So they only collect various data and upload to the trusted edge nodes directly. Examples include diverse sensors and cameras deployed within a smart factory.

Edge Nodes: Edge nodes are trusted devices with sufficient computing and communication resources, collecting data from

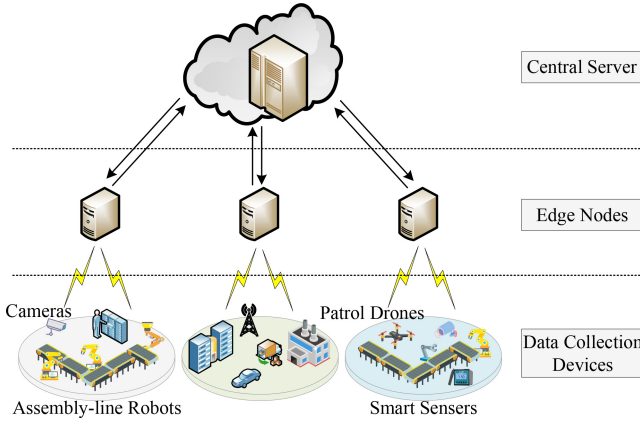


Fig. 1. System model.

various IIoT devices. Through this data and their own computational capabilities, they can perform local model training, then upload the results to the central server for aggregation. Due to diverse production lines or factory environments, data distributions across edge nodes are often non-iid.

Central Server: The resource-intensive central server generates the global model by aggregating uploaded models from edge nodes, enabling intelligent decision-making within IIoT scenarios.

As the local models shared by edge nodes and central server may reveal commercially valuable or sensitive information in IIoT scenarios, we further implement a local DP mechanism on the edge nodes to safeguard data privacy. By preventing that both the central server and edge nodes from determining whether a particular edge node participated in a communication round from shared information, this approach effectively protects the privacy of local data. To mitigate the detrimental effect of DP noise, the classifier component of the model is retained on the edge nodes, shielding it from noise perturbations. Additionally, we apply sparsification to the remaining parts of the model updates before transmission, further limiting the noise impact. Moreover, by allowing each edge node to train its own private classifier, this method efficiently reduces the accuracy decline resulting from the non-iid nature of IIoT data distributions.

V. OUR PROPOSED FEDMPS SCHEME FOR IIOT

In the traditional FedAvg algorithm, the central server aggregates the complete model, as noted in (4). However, when introducing DP, this approach leads to noise being added to all model parameters, adversely affecting model performance. Additionally, aggregating the complete model fails to sufficiently tackle the challenges presented by non-iid data. These limitations make it unsuitable to apply in the IIoT scenarios.

To overcome these limitations, we propose a novel client-level DP-based FL scheme for IIoT scenarios to mitigate the impact of noise and enhance personalization using local model partitioning and sparsification techniques. Specifically, we partition the model into a private part, which serves as

Algorithm 1 Local Model Partition

Input: Global shared model ϕ^t and local model $\omega_i^t = (\phi_i^t, h_i^t)$, private data of the i th edge node $D_i = (X_i, Y_i)$, local epochs τ_p and τ_s , learning rate η .

Output: The shared partition ϕ_i^{t+1} , the private partition h_i^{t+1} .

```

1:  $\omega_i^t \leftarrow (\phi^t, h_i^t)$ ;
2: for  $s = 1, 2, \dots, \tau_p$  do
3:    $h_i^{t+1} \leftarrow h_i^t - \eta \nabla_{h_i} f_i(\phi^t, h_i^t, D_i)$ ;
4: end for
5: for  $s = 1, 2, \dots, \tau_s$  do
6:    $\phi_i^{t+1} \leftarrow \phi_i^t - \eta \nabla_{\phi_i} f_i(\phi_i^t, h_i^{t+1}, D_i)$ ;
7: end for
8:  $\omega_i^{t+1} \leftarrow (\phi_i^{t+1}, h_i^{t+1})$ ;
9: return  $\phi_i^{t+1}, h_i^{t+1}$ .
```

the classifier, and the remaining part, which functions as a shared feature extractor. The private part is retained locally and does not participate in the aggregation process, thereby allowing for personalization while avoiding the influence of noise on the classifier. The shared part, after being updated, undergoes sparsification and is augmented with DP noise before upload.

During the local update process at edge nodes, the model is updated in two stages, as illustrated in Fig. 2. Each edge node first utilizes more informative the new shared model to update its private part (Step ①). Subsequently, the updated private model component is employed to train the shared part for later aggregation (Step ②). Upon completing this training, the shared model component is sparsified to further diminish the influence of noise (Step ③). Finally, based on the privacy budget, each edge node adds DP noise to its sparsified shared part (Step ④) before uploading it to the server for aggregation. Note that the private part itself does not receive any noise interference during this process. Further details regarding the two methods, local model partitioning and sparsification, as well as the integration of DP, are discussed in the following sections.

A. Local Model Partitioning

At first, during the local training phase, we partition the local model into two parts: 1) the final linear classifier which designated as the private part h and 2) the remaining layers which are assigned as the shared part ϕ . As a result, the local model for each edge node can be expressed as $\omega_i = (\phi_i, h_i)$, where ϕ_i is the shared feature extractor and h_i is the private classifier. The classifier h_i remains on the edge node and is not involved in the model aggregation process, while the feature extraction layer ϕ_i is shared among edge nodes and actively participates in the federated aggregation process.

The local training process for edge nodes is divided into two steps, while the complete pseudocode is visible in Algorithm 1. In communication round t , each edge node i begins by initializing their local model $\omega_i^t = (\phi^t, h_i^t)$, where ϕ^t represents the new global shared model parameters received from the server in the current training round, and h_i^t corresponds to the classifier obtained from the previous

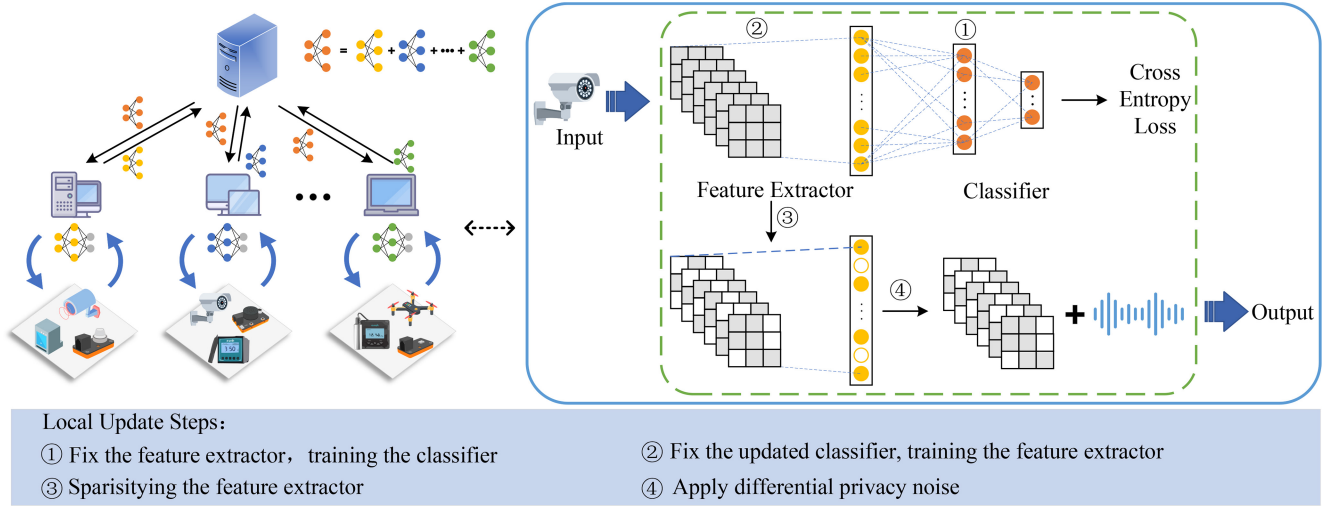


Fig. 2. Training process.

training round of i . Next, the edge node then fixes the shared part ϕ^t of the model and performs τ_l iterations to train the new classifier h_i^{t+1} (lines 2–4). For $s = 1, \dots, \tau_l$, edge node i updates its classifier as follows:

$$h_i^{t,s+1} = h_i^{t,s} - \eta_t \nabla_{h_i} f_i(\phi^t, h_i^{t,s}, D_i). \quad (7)$$

Once the new classifier h_i^t has been trained, the edge node fixes the parameters of this classifier and proceeds to train the shared part ϕ_i^t for aggregation (lines 5–7). For $s = 1, \dots, \tau_s$, the rule for updating shared model parameters can be expressed as follows:

$$\phi_i^{t,s+1} = \phi_i^{t,s} - \eta_t \nabla_{\phi_i} f_i(\phi_i^{t,s}, h_i^{t,\tau_l}, D_i). \quad (8)$$

Note that the classifier h_i^t in (8) is already trained. Upon completion of both training phases, the local model of i is updated to $\omega_i^{t+1} = (\phi_i^{t+1}, h_i^{t+1})$, where the h_i^{t+1} part remains stored locally on the edge node, while the ϕ_i^{t+1} part is included in the subsequent aggregation process, sparsification and noise addition, ensuring DP is maintained during the aggregation process.

This decision to keep the classifier local is crucial because it avoids the addition of DP noise, which could otherwise degrade its performance. Additionally, by allowing edge nodes to independently train their own classifiers, we enable better personalization of local models, improving their ability to adapt the non-iid characteristic of their private data. On the other hand, the shared feature extractor is responsible for learning general representations from the data, which are later used by the classifier for prediction. By involving this shared part in the aggregation, edge nodes can collaborate and leverage the diverse datasets from all participants, leading to the training of more robust and generalized feature extractors. This collaborative process enhances the overall model performance beyond what could be achieved by training on local data alone. In summary, this local model partitioning method improves performance by shielding the classifier from noise interference and enhances personalization through local model adaptation. Meanwhile, by sharing and aggregating

Algorithm 2 Sparsification

Input: The shared partition ϕ_i^{t+1} , compression parameter k .
Output: Sparsified parameter updates $\nabla \bar{\phi}_i^{t+1}$, the corresponding mask vector m_i^{t+1} .

- 1: $m_i^{t+1} = []$;
- 2: $\nabla \phi_i^{t+1} = \phi_i^{t+1} - \phi_i^t$;
- 3: **for** each layer l in $\nabla \phi_i^{t+1}$ **do**
- 4: select the top k coordinates of $|\nabla \phi_{i,l}^{t+1}|$ and create a corresponding mask vector $m_{i,l}^{t+1} \in \{0, 1\}^d$;
- 5: $m_i^{t+1} = m_i^{t+1} + [m_{i,l}^{t+1}]$;
- 6: **end for**
- 7: $\nabla \bar{\phi}_i^{t+1} = \nabla \phi_i^{t+1} \odot m_i^{t+1}$;
- 8: **return** $\nabla \bar{\phi}_i^{t+1}, m_i^{t+1}$.

the feature extraction layer, edge nodes achieve collaborative learning, leading to a more generalized and effective feature extractor.

B. Sparsification

After completing the training steps, edge nodes apply sparsification to the shared model parameters updates that participate in aggregation. The detailed steps are summarized in Algorithm 2. Edge node i employs a top- k sparsifier to the updates $\nabla \phi_i^{t+1}$ layer by layer. For a parameter vector $\mathbf{x} \in \mathbb{R}^d$ of a layer, the top- k sparsifier is defined as

$$[\text{top}_k(\mathbf{x})]_j := \begin{cases} [\mathbf{x}]_{\pi(j)}, & \text{if } j \leq k \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

where π is a permutation of $[d]$ such that $|[\mathbf{x}]_{\pi(j)}| \geq |[\mathbf{x}]_{\pi(j+1)}|$ for $j \in [1, d-1]$.

In this process, we retain only the top k largest update values from each layer of the shared model component, as larger values indicate that those parameters have a greater influence on the global model and should not be ignored. The remaining update values are set to zero, meaning that they will not participate in the aggregation.

Specifically, during the t th round, once edge node i completes the local training process, they obtain the model parameter updates $\nabla\phi_i^{t+1}$ to be uploaded. Then, the edge node sorts the parameter changes layer by layer and selects the top k absolute values to generate a mask vector $m_{i,l}^{t+1}$ per layer, forming a set of masks $\mathbf{m}_i^{t+1}$ (lines 1–5). Then, edge node i uses the set of masks $\mathbf{m}_i^{t+1}$ to sparsify its local model (line 6), resulting in the sparsified parameter updates $\nabla\bar{\phi}_i^{t+1}$ as follows:

$$\nabla\bar{\phi}_i^{t+1} = \nabla\phi_i^{t+1} \odot \mathbf{m}_i^{t+1} \quad (10)$$

where $\odot$ represents the Hadamard product.

Then, each layer of the shared model is sparsified by its corresponding mask, ensuring that only the most relevant parameters are retained. This sparsification process not only reduces the number of model parameters that need to be uploaded, but also diminishes the portion of the model impacted by the DP noise. As a result, the added noise has a lesser effect on the overall model performance, improving both efficiency and accuracy.

C. Differential Privacy on Sparsified Model

After the above steps, to ensure DP, edge node i only need to add noise to the effective positions of the parameter matrix set for upload (i.e., the nonzero positions in m_i^{t+1}). Since the parameter update values lack predictable boundaries, it must clip the sparsified model updates according to the formula as follows:

$$\nabla\tilde{\phi}_i^{t+1} = \nabla\bar{\phi}_i^{t+1} \times \min\left(1, C/\|\nabla\bar{\phi}_i^{t+1}\|_2\right) \quad (11)$$

where C is the clipping threshold preset.

Next, the edge node perturbs the sparsified model update values depending on the privacy budget by adding independent Gaussian noise $\mathcal{N}(0, (C^2\sigma^2/K))$ at the selected positions across each layer as follows:

$$\nabla\hat{\phi}_i^{t+1} = \nabla\tilde{\phi}_i^{t+1} + \mathcal{N}\left(0, (C^2\sigma^2/K) \cdot \mathbf{I}_d\right) \odot \mathbf{m}_i^t. \quad (12)$$

By adding DP noise, the information of each edge node is effectively obfuscated, ensuring that attackers cannot infer any participant's private data from the shared information. Furthermore, through the optimized approach we propose, local model partitioning and sparsification, the detrimental effect of noise on the model is significantly mitigated without compromising privacy protection, allowing for a better privacy–utility tradeoff.

D. Work Flow of FedMPS

Fig. 3 illustrates the workflow of the entire algorithm, which can be outlined in five main steps (①–⑤). The entire algorithm is detailed in Algorithm 3. First, the global model is initialized by the server, then distributed to all edge nodes (line 1). Federated training is then conducted over T rounds (lines 2–8). In each round, the server randomly selects K edge nodes to participate in the training (line 3). These selected edge nodes replace the shared portion of their local models with the received global shared model. They then perform local training, updating the private and shared portions

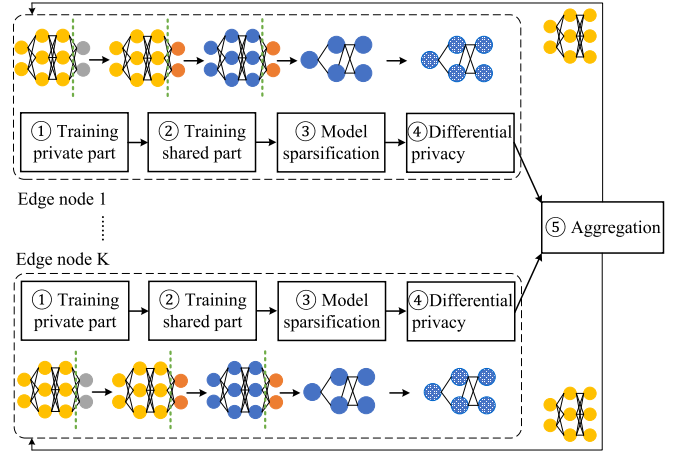


Fig. 3. Work flow of FedMPS.

Algorithm 3 FedMPS

Input: Number of edge nodes N , number of selected edge node per round K , number of communication rounds T , local epochs τ_p and τ_s , learning rate η , clipping threshold C , privacy budget (ϵ, δ) , compression parameter k .
Output: Global shared model ϕ^T , local personalized models $h_1^T, h_2^T, \dots, h_N^T$.

Server:

- 1: Initializes a global model $\omega^0 \in \mathbb{R}^d$ and distributes to all edge nodes;
- 2: **for** $t = 0, 1, \dots, T - 1$ **do**
- 3: Select K edge nodes randomly and broadcast ϕ^t to all selected edge nodes;
- 4: **for** each edge node $i \in [1, K]$ in parallel **do**
- 5: $\hat{\phi}_k^{t+1} = \text{Client}_i(\phi^t)$;
- 6: **end for**
- 7: $\phi^{t+1} = \sum_{i \in K} \frac{n_i}{n} \hat{\phi}_k^{t+1}$;
- 8: **end for**

Client_i(ϕ^t):

- 9: $\phi_i^{t+1}, h_i^{t+1} = \text{Algorithm 1}$; // Local Training
- 10: $\nabla\bar{\phi}_i^{t+1}, \mathbf{m}_i^{t+1} = \text{Algorithm 2}$; // Sparsification
- 11: $\nabla\tilde{\phi}_i^{t+1} = \nabla\bar{\phi}_i^{t+1} \times \min(1, C/\|\nabla\bar{\phi}_i^{t+1}\|_2)$; // Clipping Local Updates
- 12: $\nabla\hat{\phi}_i^{t+1} = \nabla\tilde{\phi}_i^{t+1} + \mathcal{N}(0, (C^2\sigma^2/K) \cdot \mathbf{I}_d) \odot \mathbf{m}_i^{t+1}$; // Adding Gaussian Noise
- 13: **return** $\hat{\phi}_k^{t+1}$.

of their models separately, as outlined in Algorithm 1 (line 9). Subsequently, sparsification is applied, as described in Algorithm 2 (line 10). Finally, noise for DP is added to the compressed shared model parameters (lines 11 and 12), as shown in (11) and (12). These parameters are then uploaded to the server, which aggregates the parameters to create a new global shared model, and redistributes it to the edge nodes in the next training round (line 7).

VI. PRIVACY ANALYSIS

This section conducts a complete privacy analysis of FedMPS leveraging RDP as the foundation for our evaluation. Let $\mathcal{W}$ and $\mathcal{W}'$ be two sets of clients that differ by exactly one client, denoted as c , such that $\mathcal{W}' := \mathcal{W} \cup \{c\}$. According to the definition of l_2 -sensitivity in the preliminary section, the l_2 -sensitivity of the sum of locally shared model updates can be formulated as follows:

$$\psi(h) = \max \left\| \sum_{i \in \mathcal{W}} \Delta_i^t(D_i) - \sum_{j \in \mathcal{W}'} \Delta_j^t(D_j) \right\|_2. \quad (13)$$

As a result of the clipping operation, we obtain the following bound $\|\Delta_i^t(D_i)\|_2 \leq C \forall i \in [n]$. Thus, the l_2 -sensitivity $\psi(h) = \max \|\Delta_c^t(D_c)\|_2 \leq C$ which is bounded by C . Since the sum of Gaussian random variables remains Gaussian random property, the final privacy guarantee for FedMPS can be computed as follows, based on the variance of the Gaussian noise $C^2\sigma^2$ introduced into the sum of uploaded updates and the results from the lemmas mentioned in the preliminary section.

Theorem 1: Assume that clients are sampled without replacement, with a sampling rate of $q = K/N$ per round. For any given $\epsilon < 2\log(1/\delta)$ and $\delta \in (0, 1)$, after T communication rounds, FedMPS satisfies DP if the following condition holds:

$$\sigma^2 \geq \frac{7q^2T(\epsilon + 2\log(1/\delta))}{\epsilon^2}. \quad (14)$$

Proof: Once noise $\mathcal{N}(0, C^2\delta^2)$ is added to the K retained positions of the sparsified local updates, the sum of these updates for participating clients in each round satisfies $(\alpha, \alpha/2\delta^2)$ -RDP, as shown the definition of Gaussian mechanism. Since the K clients are uniformly sampled in each round from the full set of clients, the subsampling amplification property of RDP, as outlined in the RDP subsampling lemma, further enhances the privacy guarantee of FedMPS for each round. After T rounds, by applying the composition property of RDP, as detailed in the RDP composition lemma, the overall privacy guarantee can be derived. Finally, by using the translation lemma in preliminary section, we can convert the RDP guarantee into the standard (ϵ, δ) -DP guarantee. ■

VII. EXPERIMENTS AND ANALYSIS

This section provides a detailed analysis of comparative and ablation experiments conducted on public datasets to showcase the effectiveness of our proposed method.

A. Experiment Setup

Scenario and Datasets Settings: We construct a non-iid scenario to simulate the heterogeneous data collection across various end devices in IIoT applications. To evaluate our proposed scheme, we employ three widely recognized datasets. The first is Fashion-MNIST, which includes 70 000 grayscale 28×28 pixel images of ten clothing categories. The second dataset is CIFAR-10, which contains 60 000 color 32×32 pixel images across ten classes. Additionally, we include CIFAR-100 to further validate our approach on more complex datasets. This

dataset also contains 60 000 color images of size 32×32 pixels, but these are divided into 100 fine-grained classes. Each class contains 600 images. This dataset presents greater challenges due to its finer categorization and increased heterogeneity. For all three datasets, 10 000 samples are reserved for testing, with the remaining samples allocated for training.

To simulate the data heterogeneity characteristic of IIoT applications, we use the Dirichlet distribution $\text{Dir}(\alpha)$ to perform non-iid partitioning on both datasets, where the parameter α determines the extent of data heterogeneity. When α is smaller, the local data distribution of each edge node tends to differ more significantly from the overall dataset. We test the performance of our method and baselines under different values of α , with detailed parameter settings provided in the subsequent hyperparameters section.

Model Setting: For the Fashion-MNIST and CIFAR-10 datasets, we employ a convolutional neural network (CNN) model. The architecture comprises two convolutional layers, with the first applying 32 filters of size 5×5 and the second using 64 filters of the same size. Each layer is followed by a 2×2 max pooling layer, with ReLU activation functions applied afterward. After the preceding layers, the output is flattened into a single vector and forwarded to a fully connected layer consisting of 128 neurons for Fashion-MNIST or 512 neurons for CIFAR-10, where a ReLU activation function is also applied. Finally, the network ends with a softmax output layer, to generate probabilities for classification tasks. For the more complex CIFAR-100 dataset, we use a deeper ResNet-18 model for training. The architecture begins with a convolutional layer, followed by eight residual blocks, each containing two 3×3 convolutional layers, batch normalization, and ReLU activation. The final classification layer consists of a fully connected layer followed by softmax to predict the probabilities for each of the 100 classes.

Baselines: We utilize four baseline methods for comparison to illustrate how effective our method is. The first baseline is the original DP-FedAvg [8], which directly applies DP noise to the entire local model without any optimization techniques to mitigate the noise's impact on model performance. The second baseline is BLUR-LUS [10], which improves upon the loss function by incorporating a regularization term related to the clipping threshold. This method further enhances model robustness by applying local model sparsification, effectively reducing the norm of local updates, which consequently reduces the negative impact of DP noise. The third baseline is FedSMP [13], which introduces a global sparsification technique. Before noise is added to the local model, a unified sparsification is applied, reducing the dimensionality of the local model updates and consequently limiting the detrimental effect of the noise on the model's overall performance. Finally, we compare against FedDPA [25], which enhances robustness by fixing personalized parameters locally using a Fisher threshold to determine which parts of the model are retained, reducing the impact of noise.

Hyperparameters: In the experiments, we set the batch size to 64, and the datasets are distributed across 50 clients based on a Dirichlet distribution with $\alpha = [0.1, 0.3, 0.7, 1.0]$. Randomly, the server selects $K = [1, 5, 10, 20, 50]$ clients to

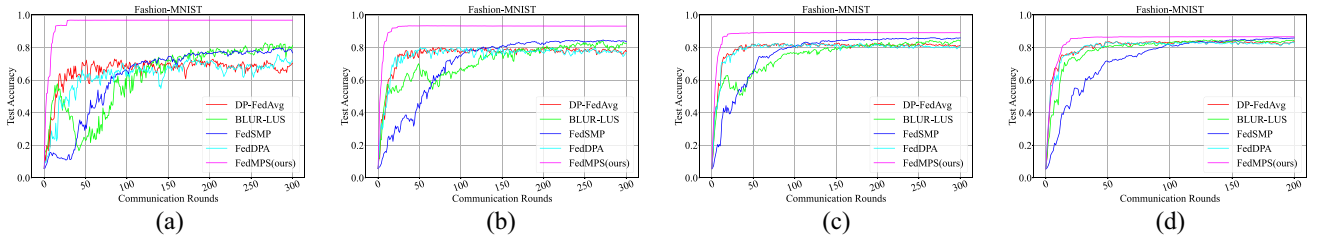


Fig. 4. Performance of different algorithms under different data heterogeneity on Fashion-MNIST. (a) Performance of different algorithms on Fashion-MNIST ($\alpha = 0.1$). (b) Performance of different algorithms on Fashion-MNIST ($\alpha = 0.3$). (c) Performance of different algorithms on Fashion-MNIST ($\alpha = 0.7$). (d) Performance of different algorithms on Fashion-MNIST ($\alpha = 1.0$).

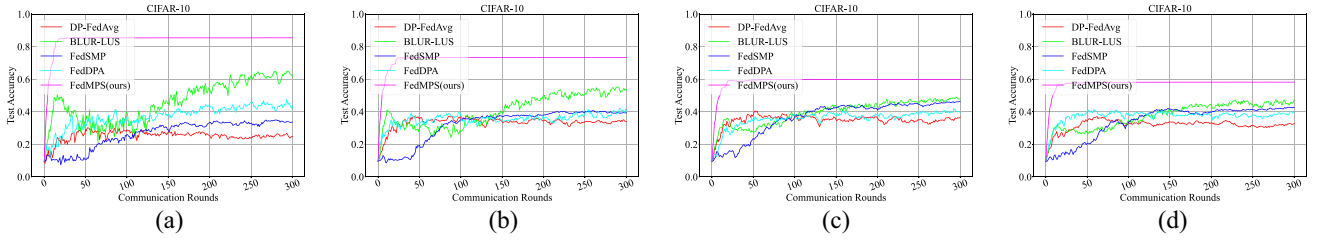


Fig. 5. Performance of different algorithms under different data heterogeneity on CIFAR-10. (a) Performance of different algorithms on CIFAR-10 ($\alpha = 0.1$). (b) Performance of different algorithms on CIFAR-10 ($\alpha = 0.3$). (c) Performance of different algorithms on CIFAR-10 ($\alpha = 0.7$). (d) Performance of different algorithms on CIFAR-10 ($\alpha = 1.0$).

TABLE I
HYPERPARAMETERS IN OUR EXPERIMENTS

Hyperparameters	Meaning	Value
N	Number of clients	20/50
K	Number of selected clients	1/5/10/20/50
T	Communication rounds	200/300
E	Local epochs	5
α	Dirichlet parameter	0.1/0.3/0.7/1.0
B	Batch size	64
η	Learning rate	0.03
τ_p	Local epochs for h_i	3
τ_s	Local epochs for ϕ_i	5
k	Compression rate	0.1/0.3/0.5/0.7/0.9
ϵ	privacy budget	2/4/6

participate in the training process during each communication round. Each client performs 5 local training epochs with a learning rate of 0.03, using SGD as the optimizer. For the Fashion-MNIST dataset, the total number of communication rounds is set to 200, while for CIFAR-10 and CIFAR-100, it is increased to 300 due to the higher complexity of the task. Regarding the parameters for the DP algorithm, we experiment with three privacy budgets $\epsilon = [2, 4, 6]$. The value of δ is set to the reciprocal of the number of users (i.e., $1/N$), as proposed in [25]. Additionally, the clipping threshold is set to $C = 0.2$ to ensure that the gradients remain bounded, which is the optimal value obtained from testing various values of $C = [0.1, 0.2, 0.3, 0.4, 0.5]$.

Table I provides a summary of the hyperparameters used in our experiments. Moreover, for the baseline methods BLUR-LUS and FedSMP, we set the compression rate k to 0.3 (i.e., 30% of the model parameters are retained during sparsification). We also investigated the impact of different compression rates ($k = [0.1, 0.3, 0.5, 0.7, 0.9]$) on our method. In the case of BLUR, the regularization coefficient is set to 0.2, which is the optimal value selected from the original paper's

parameter settings. Then, for FedDPA, the hyperparameters are set consistent with the original paper, with the Fisher threshold set to 0.2, and the regularization coefficients set to 0.1 and 0.05.

B. Experimental Results

1) *Comparison Experiments*: To showcase the superiority of our proposed method in the context of DP and non-iid data, we conducted a comparison of test accuracy against four baseline approaches, using both the Fashion-MNIST and CIFAR-10 datasets. These datasets were sampled from a Dirichlet distribution with four distinct parameters, $\alpha = [0.1, 0.3, 0.7, 1.0]$, to generate varying degrees of data heterogeneity. Furthermore, to validate the effectiveness of our method on more complex datasets, we also conducted experiments on the CIFAR-100 dataset. The privacy budget was fixed at $\epsilon = 4$ for each experiment.

As shown in Fig. 4(a)–(d) and the columns from 2 to 5 of Table II, on the Fashion-MNIST dataset, our method outperforms all other baselines in terms of both accuracy and convergence speed, achieving a 0.7%–3.5% improvement in accuracy over the alternatives. The original DP-FedAvg [8] method, which applies no optimizations, exhibits lower accuracy compared to the others. Similarly, Fig. 5(a)–(d) and the columns from 6 to 9 of Table II present the results on the more complex CIFAR-10 dataset, where our method also demonstrates better performance under stringent privacy guarantees, with a 10.7%–21.2% improvement in accuracy over the baselines. In addition, it achieves the fastest convergence speed and shows greater robustness to the added noise compared to the other approaches. Notably, due to the smaller privacy budget used in our experiments, which leads to more noise being added, the FedDPA [25] method experiences a drop in accuracy compared to its original paper.

TABLE II
BEST ACCURACY (%) OF DIFFERENT ALGORITHMS

Method	Fashion-MNIST				CIFAR-10				CIFAR-100			
	$\alpha = 0.1$	$\alpha = 0.3$	$\alpha = 0.7$	$\alpha = 1.0$	$\alpha = 0.1$	$\alpha = 0.3$	$\alpha = 0.7$	$\alpha = 1.0$	$\alpha = 0.1$	$\alpha = 0.3$	$\alpha = 0.7$	$\alpha = 1.0$
DP-FedAvg [8]	72.98	80.23	82.78	83.10	30.13	37.24	40.42	36.97	3.54	-	-	-
BLUR-LUS [10]	82.99	84.14	84.60	84.75	64.88	55.32	49.11	47.44	-	-	-	-
FedSMP [13]	79.94	84.70	86.06	85.97	34.98	40.69	46.31	42.81	-	-	-	-
FedDPA [25]	75.84	80.11	82.82	83.73	47.38	42.32	41.91	41.55	-	-	-	-
FedMPS (ours)	96.84	93.35	89.22	86.62	85.55	73.41	59.88	58.19	47.92	34.76	25.60	23.12

TABLE III
COMPARISON OF ACCURACY (%) ON FASHION-MNIST AND CIFAR-10 UNDER DIFFERENT PRIVACY BUDGETS

Method	Fashion-MNIST			CIFAR-10		
	$\epsilon = 2$	$\epsilon = 4$	$\epsilon = 6$	$\epsilon = 2$	$\epsilon = 4$	$\epsilon = 6$
DP-FedAvg [8]	81.76	83.10	83.78	32.31	36.97	37.66
BLUR-LUS [10]	83.57	84.75	85.61	47.88	47.44	47.95
FedSMP [13]	85.50	85.97	86.69	42.56	42.81	45.62
FedDPA [25]	82.80	83.73	84.52	37.97	41.55	43.34
FedMPS(Ours)	86.66	86.62	86.73	58.02	58.19	57.91

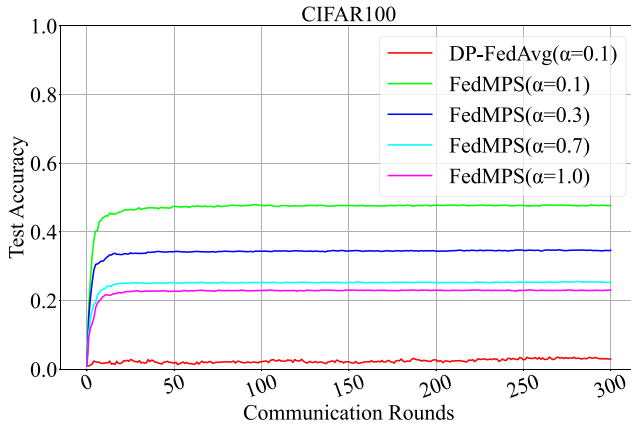


Fig. 6. Performance on CIFAR-100 with different α .

Additionally, when comparing the first two datasets, we observe that the performance differences between algorithms are smaller on Fashion-MNIST. This is because Fashion-MNIST is relatively simple, allowing each method to perform well even with added DP noise and heterogeneous data. In contrast, this performance gap becomes more pronounced on the more challenging CIFAR-10 dataset, where the complexity of the task highlights the advantages of our approach.

For the CIFAR-100 dataset, we employ a deeper model ResNet-18 during training. To avoid issues arising from each edge node having too few samples of each class, we set the total number of edge nodes to 20 and randomly select half of them for each communication round. As shown in Fig. 6 and the columns from 10 to 13 of Table II, our method remains effective with increased complexity. However, due to the larger number of model parameters, the negative impact of noise becomes more pronounced, and the complexity of the dataset also contributes to a significant performance degradation in the other baseline methods, making their accuracies dropping below 5%. As a result, we omitted their results in the figure and table. This further demonstrates that our method can effectively mitigate the interference of noise on model performance and is capable of handling more complex models.

From the perspective of data heterogeneity, our method demonstrates higher accuracy and faster convergence across all values of α , indicating the effectiveness of our method in handling varying levels of data distribution skewness. Additionally, our approach exhibits strong robustness. When α is smaller, the performance improvement of our method over the baseline approaches is more significant.

2) *Performance Under Different Privacy Budgets:* Table III presents the test accuracy of different algorithms on the Fashion-MNIST and CIFAR-10 datasets under varying privacy budgets, fixing the Dirichlet parameter α to 1. Our method consistently outperforms all baseline algorithms, achieving a superior utility–privacy tradeoff. Specifically, compared to the original DP-FedAvg [8], our approach shows an improvement of 20.3%–25.7% in test accuracy on CIFAR-10. Similarly, when compared to other baseline methods [10], [13], [25], our algorithm exhibits a 10.2%–16.6% accuracy increase. On the Fashion-MNIST dataset, however, as the dataset is relatively simple, all methods achieve high accuracy under three privacy budgets. Consequently, the accuracy improvement achieved by our method is less pronounced compared to CIFAR-10, with gains of 1.1%–3.8%.

Furthermore, we observe that under smaller privacy budgets (i.e., stricter privacy constraints), the gap between our method and the baseline methods becomes larger, which suggests that our method is particularly well suited for applications requiring stronger privacy protection, offering better utility while maintaining rigorous privacy guarantees.

3) *Ablation Experiments:* In order to demonstrate the effectiveness of the two key techniques in our method, model partitioning and sparsification, we conducted ablation studies to assess their individual contributions. These experiments were performed on the CIFAR-10 dataset with a fixed privacy budget of $\epsilon = 2$. The results are presented in Fig. 7 and Table IV, where “FedMPS-mp” denotes the model partitioning-only configuration, and “FedMPS-s” represents the sparsification-only configuration. As shown in the results, both techniques independently improve performance compared

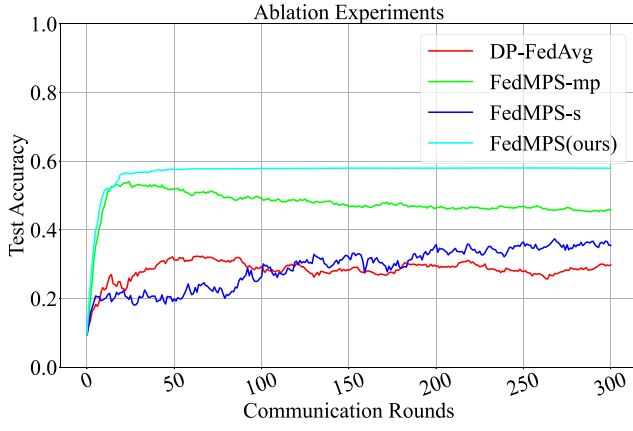


Fig. 7. Ablation experiments.

TABLE IV
BEST ACCURACY OF ABLATION EXPERIMENTS

Method	Accuracy (%)	Increase
DP-FedAvg [8]	32.31	-
FedMPS-mp	54.08	21.77% ↑
FedMPS-s	37.38	5.07% ↑
FedMPS(ours)	58.02	25.71% ↑

to the baseline method DP-FedAvg [8]. Specifically, when using model partitioning alone, there is a significant improvement in test accuracy, increasing by 22% as it avoiding the interference to the classification and bringing additional personalization ability. However, this approach is not robust enough to DP noise, which accumulates over multiple training rounds, causing a decline in model performance. On the other hand, when sparsification is applied alone, it markedly enhances the model's robustness by limiting the impact of the noise to a smaller portion of the model parameters, and also resulting in a 5% accuracy improvement. By combining these two techniques, our method effectively reduces the noise's detrimental impact on model performance. This allows us to achieve higher accuracy while maintaining strong privacy guarantees, and adapts to non-iid settings as well.

4) *Different Compression Rates:* We investigate the impact of different compression rates on our method using the CIFAR-10 dataset with Dirichlet parameter $\alpha = 1$ and varying privacy budgets. The results are presented in Table V. As shown in the table, there exists an optimal compression rate k , which varies with the privacy budget. This is due to the tradeoff between compression and DP noise. When k is smaller, fewer parameters are uploaded, reducing the impact of noise on the model. However, this comes at the cost of increased performance degradation due to higher sparsification. Conversely, when k is larger, although sparsification is reduced, the impact of noise increases. Specifically, when $\epsilon = 2$, the noise effect is more significant, and the optimal k is found to be 0.1. As the privacy budget increases to $\epsilon = 4$ or $\epsilon = 6$, the optimal k increases to 0.3, reflecting a shift toward balancing the effects of noise and compression loss.

5) *Different Numbers of Selected Clients:* Table VI shows the performance of our method with different numbers of selected edge nodes in each communication round (i.e., K).

TABLE V
BEST ACCURACY UNDER DIFFERENT COMPRESSION RATES AND PRIVACY BUDGETS

Compression Rate	Accuracy (%)		
	$\epsilon = 2$	$\epsilon = 4$	$\epsilon = 6$
$k = 0.1$	58.14	58.28	58.03
$k = 0.3$	58.02	58.01	58.15
$k = 0.5$	57.97	58.52	58.55
$k = 0.7$	57.29	57.75	58.16
$k = 0.9$	57.17	57.35	58.15

TABLE VI
BEST ACCURACY AND CONVERGENCE ROUND UNDER DIFFERENT NUMBERS OF SELECT CLIENTS AND PRIVACY BUDGETS

Number	Accuracy (%)			Convergence Round		
	$\epsilon = 2$	$\epsilon = 4$	$\epsilon = 6$	$\epsilon = 2$	$\epsilon = 4$	$\epsilon = 6$
$K = 1$	57.32	57.37	54.64	279	274	251
$K = 5$	58.20	58.36	58.28	122	100	82
$K = 10$	58.02	58.19	57.91	54	56	59
$K = 20$	58.15	58.41	58.16	25	24	25
$K = 50$	58.19	58.19	58.09	9	10	10

This experiment is also conducted on the CIFAR-10 dataset with Dirichlet parameter $\alpha = 1$. In terms of accuracy, the number of participating edge nodes has a minimal impact, demonstrating the robustness of our method. However, the convergence round is still influenced by this number. The more edge nodes involved in each training round, the more data information can be collaboratively utilized, leading to faster convergence. For instance, when all edge nodes participate in each round, the model converges within 20 rounds. In contrast, when only one edge node participates in each round, the convergence is much slower, requiring more than 250 rounds. Additionally, different privacy budgets also have an effect on convergence. Stronger privacy protection, which results in larger noise addition, leads to slower convergence, particularly when few edge nodes participate in each round.

VIII. CONCLUSION

In this article, to achieve a improved tradeoff between privacy and utility in heterogeneous IIoT scenarios, we propose a robust DPFL method FedMPS. By partitioning the local model into shared and private sections, along with applying sparsification before adding DP noise, our method effectively mitigates the detrimental effect of noise on model performance without compromising privacy guarantees, resulting in a better privacy-utility tradeoff. Furthermore, our method is robust in non-iid settings, tackling the challenges arising from the heterogeneous data of end devices. Theoretical analysis shows that our method offers DP protection. Compared with four baselines, experimental results demonstrate the effectiveness of our method.

REFERENCES

- [1] P. Boobalan et al., "Fusion of federated learning and Industrial Internet of Things: A survey," *Comput. Netw.*, vol. 212, Jul. 2022, Art. no. 109048.
- [2] V. K. Quy, D. C. Nguyen, D. Van Anh, and N. M. Quy, "Federated learning for green and sustainable 6G IIoT applications," *Internet Things*, vol. 25, Apr. 2024, Art. no. 101061.

- [3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Stat.*, 2017, pp. 1273–1282.
- [4] S. Ali, Q. Li, and A. Yousafzai, "Blockchain and federated learning-based intrusion detection approaches for edge-enabled industrial IoT networks: A survey," *Ad Hoc Netw.*, vol. 152, Jan. 2024, Art. no. 103320.
- [5] R. Shen et al., "BAFL-SVM: A blockchain-assisted federated learning-driven SVM framework for smart agriculture," *High-Confidence Comput.*, vol. 5, no. 1, Mar. 2025, Art. no. 100243.
- [6] D. Yu, H. Zhang, Y. Huang, and Z. Xie, "Data distribution inference attack in federated learning via reinforcement learning support," *High-Confidence Comput.*, vol. 5, no. 1, 2025, Art. no. 100235.
- [7] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, nos. 3–4, pp. 211–407, 2014.
- [8] R. C. Geyer, T. Klein, and M. Nabi, "Differentially private federated learning: A client level perspective," 2017, *arXiv:1712.07557*.
- [9] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–14.
- [10] A. Cheng, P. Wang, X. S. Zhang, and J. Cheng, "Differentially private federated learning with local regularization and sparsification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10122–10131.
- [11] X. Shen, Y. Liu, and Z. Zhang, "Performance-enhanced federated learning with differential privacy for Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 24079–24094, Dec. 2022.
- [12] Y. Shi, Y. Liu, K. Wei, L. Shen, X. Wang, and D. Tao, "Make landscape flatter in differentially private federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 24552–24562.
- [13] R. Hu, Y. Guo, and Y. Gong, "Federated learning with sparsified model perturbation: Improving accuracy under client-level differential privacy," *IEEE Trans. Mobile Comput.*, vol. 23, no. 8, pp. 8242–8255, Aug. 2024.
- [14] Y. Yang, B. Hui, H. Yuan, N. Gong, and Y. Cao, "PrivateFL: Accurate, differentially private federated learning via personalized data transformation," in *Proc. 32nd USENIX Security Symp. (USENIX Security)*, 2023, pp. 1595–1612.
- [15] G. Xu, Z. Zhou, J. Dong, L. Zhang, and X. Song, "A blockchain-based federated learning scheme for data sharing in Industrial Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21467–21478, Dec. 2023.
- [16] M. A. P. Putra, R. N. Alief, S. M. Rachmawati, G. A. Sampedro, D.-S. Kim, and J.-M. Lee, "Proof-of-authority-based secure and efficient aggregation with differential privacy for federated learning in industrial IoT," *Internet Things*, vol. 25, Apr. 2024, Art. no. 101107.
- [17] X. Tan, D. Xiao, H. Huang, M. Wang, and M. Li, "Hierarchically fair and differentially private federated learning in industrial IoT based on compressed sensing with adaptive-thresholding sparsification," *IEEE Trans. Ind. Informat.*, vol. 20, no. 12, pp. 13731–13740, Dec. 2024.
- [18] Z. Li et al., "Data heterogeneity-robust federated learning via group client selection in industrial IoT," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 17844–17857, Sep. 2022.
- [19] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2016, pp. 308–318.
- [20] G. Andrew, O. Thakkar, B. McMahan, and S. Ramaswamy, "Differentially private learning with adaptive clipping," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 17455–17466.
- [21] X. Zhang, F. Yang, Y. Guo, H. Yu, Z. Wang, and Q. Zhang, "Adaptive differential privacy mechanism based on entropy theory for preserving deep neural networks," *Mathematics*, vol. 11, no. 2, p. 330, 2023.
- [22] Y. Miao, R. Xie, X. Li, X. Liu, Z. Ma, and R. H. Deng, "Compressed federated learning based on adaptive local differential privacy," in *Proc. 38th Annu. Comput. Security Appl. Conf.*, 2022, pp. 159–170.
- [23] Z. He, L. Wang, and Z. Cai, "Clustered federated learning with adaptive local differential privacy on heterogeneous IoT data," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 137–146, Jan. 2024.
- [24] A. Bietti, C.-Y. Wei, M. Dudik, J. Langford, and S. Wu, "Personalization improves privacy-accuracy tradeoffs in federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 1945–1962.
- [25] X. Yang, W. Huang, and M. Ye, "Dynamic personalized federated learning with adaptive differential privacy," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 72181–72192.
- [26] T. T. Phuong and L. T. Phong, "Differentially private stochastic gradient descent via compression and memorization," *J. Syst. Archit.*, vol. 135, Feb. 2023, Art. no. 102819.
- [27] R. Kerkouche, G. Ács, C. Castelluccia, and P. Genevès, "Compression boosts differentially private federated learning," in *Proc. IEEE Eur. Symp. Security Privacy (EuroSP)*, 2021, pp. 304–318.
- [28] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 6357–6368.
- [29] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 10713–10722.
- [30] Y. Huang et al., "Personalized cross-silo federated learning on non-iid data," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 7865–7873.
- [31] J. Zhang et al., "FedALA: Adaptive local aggregation for personalized federated learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, 2023, pp. 11237–11244.
- [32] L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, "Exploiting shared representations for personalized federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 2089–2099.
- [33] J. H. Oh, S. Kim, and S. Yun, "FedBABU: Toward enhanced representation for federated image classification," in *Proc. 10th Int. Conf. Learn. Represent.*, 2022, pp. 1–29.
- [34] L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, "FedAvg with fine tuning: Local updates lead to representation learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 10572–10586.
- [35] Y. Tan et al., "FedProto: Federated prototype learning across heterogeneous clients," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, 2022, pp. 8432–8440.
- [36] L. Yi, G. Wang, X. Liu, Z. Shi, and H. Yu, "FedGH: Heterogeneous federated learning with generalized global header," in *Proc. 31st ACM Int. Conf. Multimedia*, 2023, pp. 8686–8696.
- [37] I. Mironov, "Rényi differential privacy," in *Proc. IEEE 30th Comput. Security Found. Symp. (CSF)*, Los Alamitos, CA, USA, 2017, pp. 263–275.
- [38] Y.-X. Wang, B. Balle, and S. P. Kasiviswanathan, "Subsampled Rényi differential privacy and analytical moments accountant," in *Proc. 22nd Int. Conf. Artif. Intell. Stat.*, 2019, pp. 1226–1235.